

AI Pipeline – What Did You Understand?

Part 1: What Did You Understand?

1. What is an AI Pipeline?

An AI Pipeline is a sequence of steps used to build, train, evaluate, and use an Artificial Intelligence or Machine Learning system.

It organizes the whole AI project into clear stages. Each stage has a specific purpose, starting from collecting data and preparing it, then training the AI model, testing its performance, and finally using the model in a real application.

2. What are the main steps of an AI Pipeline?

The main steps of a typical AI Pipeline are:

1. **Data Collection**
 2. **Data Cleaning**
 3. **Data Preprocessing**
 4. **Feature Engineering**
 5. **Model Training**
 6. **Model Evaluation**
 7. **Deployment**
 8. **Monitoring and Improvement**
-

3. What happens in each step?

1. Data Collection

In this step, we collect the data needed for the AI project.

The data can come from different sources such as databases, websites, sensors, applications, or existing datasets.

Example: For a spam detection system, we collect emails that are labeled as spam or not spam.

2. Data Cleaning

The collected data may contain missing values, duplicates, incorrect information, or unnecessary data.

In this step, we clean the data and remove or correct these problems.

Example: We can remove duplicate emails and handle emails with missing information.

3. Data Preprocessing

The data is prepared in a format that the AI model can understand.

Depending on the project, this may include converting data into numerical values, normalizing values, encoding categories, or preparing text and images.

Example: Email text can be converted into numerical features that a machine learning model can process.

4. Feature Engineering

Features are the important pieces of information that help the model make predictions.

In this step, we select useful features or create new ones from the available data.

Example: For spam detection, useful features could include the number of suspicious words, the number of links, or the length of the email.

5. Model Training

In this stage, the AI model learns patterns from the prepared training data.

The model uses the input features and known answers to learn how to make predictions.

Example: The model learns the patterns that are common in spam emails.

6. Model Evaluation

After training, we test the model using data that it has not seen before.

We use evaluation metrics such as accuracy, precision, recall, or F1-score to determine how well the model performs.

Example: We check how accurately the model can identify new emails as spam or non-spam.

7. Deployment

If the model performs well enough, it can be deployed into a real application or system.

This allows users or other applications to use the model's predictions.

Example: The trained spam detection model can be integrated into an email application to automatically classify incoming emails.

8. Monitoring and Improvement

After deployment, the model should be monitored to make sure it continues to work correctly.

New data can be collected and the model may need to be updated or retrained when its performance decreases.

Example: New types of spam emails may appear, so the model can be retrained using newer email data.

4. Why do we need an AI Pipeline?

We need an AI Pipeline because building an AI system is not only about training a model.

A pipeline organizes the entire process and makes it easier to manage, test, update, and improve the AI system.

It also helps ensure that the data is properly prepared and that the model is evaluated before it is used in a real application.

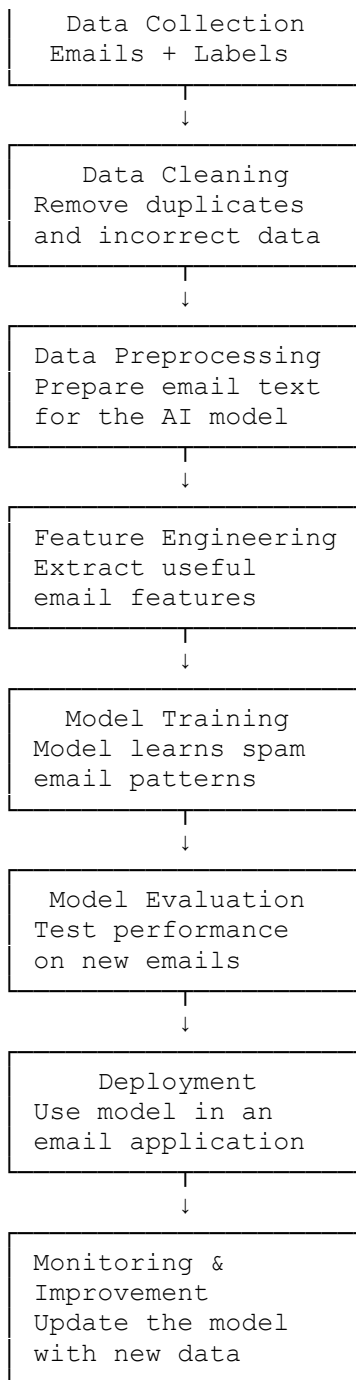
Without a clear pipeline, the project can become difficult to manage and errors can occur at different stages.

Part 2: Simple Example

Chosen AI Project: Spam Email Detection

For this example, the goal is to build an AI system that can automatically classify an email as either **Spam** or **Not Spam**.

AI Pipeline Diagram



Explanation of the Example

First, we collect a dataset containing many emails labeled as spam or not spam.

Then, we clean the data by removing duplicates and incorrect or unnecessary information.

After that, the email text is preprocessed so that it can be understood by the machine learning model.

Next, important features are extracted from the emails, such as suspicious words, number of links, and email length.

The model is then trained using the prepared data so that it can learn the patterns of spam emails.

After training, the model is evaluated using new emails to measure its performance.

If the model performs well, it is deployed into an email application where it can classify incoming emails.

Finally, the system is monitored continuously. If new types of spam appear or the model's performance decreases, new data can be used to improve or retrain the model.

Conclusion

An AI Pipeline provides an organized process for developing an AI system. It starts with collecting and preparing data, continues with training and evaluating the model, and ends with deployment and continuous improvement.

For a spam email detection system, the pipeline allows the AI model to learn from existing emails and then use what it learned to classify new emails automatically.

Contents

Task 1 – ERD vs ARD vs EERD

Task 2 – Database Structures

Task 3 – SQL Advantages & Disadvantages

Task 4 – OLTP vs OLAP

Task 5 – The 20 Vs of Big Data

Task 6 – Data Engineering Tools Research

Task 1: ERD vs ARD vs EERD

1.1 ERD – Entity Relationship Diagram

An Entity Relationship Diagram (ERD) is a visual model that shows the main entities in a system, their attributes, and the relationships between them. It is mainly used during database analysis and design to understand what information the database needs to store and how different pieces of information are connected.

Main components:

- Entities: important objects or concepts, such as Student, Course, Order, or Product.
- Attributes: properties that describe an entity, such as Student_ID, Name, or Email.
- Relationships: associations between entities, such as Student ENROLLS IN Course.
- Keys: attributes used to identify records, especially primary keys and foreign keys.
- Cardinality: rules describing one-to-one, one-to-many, or many-to-many relationships.

Typical level of detail: conceptual to logical. ERDs give a clear overview without requiring physical database implementation details.

1.2 ARD – Attribute Relationship Diagram

An Attribute Relationship Diagram (ARD) is an attribute-focused representation used in some database-design curricula to show attributes and the relationships/dependencies between them. The term ARD is less standardized than ERD and EERD, so its exact notation can vary by course or textbook. For this assignment, ARD is treated as a more attribute-centric view of the same data model.

Main components:

- Attributes and their names/data meanings.
- Relationships or dependencies between attributes and the entities/tables they belong to.
- Identifiers/keys and references between related data items.
- Optionality or dependency information when required by the modeling notation.

Typical level of detail: logical/detail-oriented. It is useful when moving from a conceptual model toward table and column definitions.

1.3 EERD – Enhanced Entity Relationship Diagram

An Enhanced Entity Relationship Diagram (EERD) extends the normal ER model with additional concepts needed for more complex systems. Common concepts include superclasses and subclasses, specialization, generalization, inheritance, and categories.

Main components:

- All normal ERD components: entities, attributes, relationships, keys, and cardinalities.
- Superclasses and subclasses.
- Specialization: creating more specific subtypes from a general entity.
- Generalization: combining similar subtypes into a more general superclass.
- Inheritance of common attributes and relationships.
- Constraints such as disjoint/overlapping and total/partial specialization.

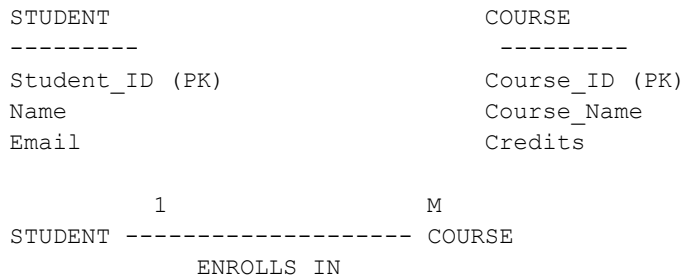
1.4 Main Differences

Aspect	ERD	ARD	EERD
Main focus	Entities and relationships	Attributes and their relationships/dependencies	Entities + advanced semantic modeling
Complexity	Low to medium	Medium/detail-oriented	Medium to high
Main elements	Entities, attributes, relationships, keys	Attributes, identifiers, dependencies	ERD elements + subtypes/supertypes/inheritance
Level of detail	Conceptual/logical overview	More attribute-level detail	Detailed logical/conceptual model
Best use	Initial database design	Detailed attribute analysis	Complex systems with different entity subtypes

1.5 Practical Example – Student Management System

The same Student Management System can be represented differently depending on the diagram.

A. ERD representation



Enrollment relationship:
Student_ID + Course_ID + Enrollment_Date

The ERD focuses on the main entities and how they relate. Because a student can enroll in many courses and a course can have many students, the many-to-many relationship is normally resolved with an associative entity such as Enrollment.

B. ARD representation

```
STUDENT attributes
Student_ID -> identifies Student
Name       -> describes Student
Email      -> describes Student

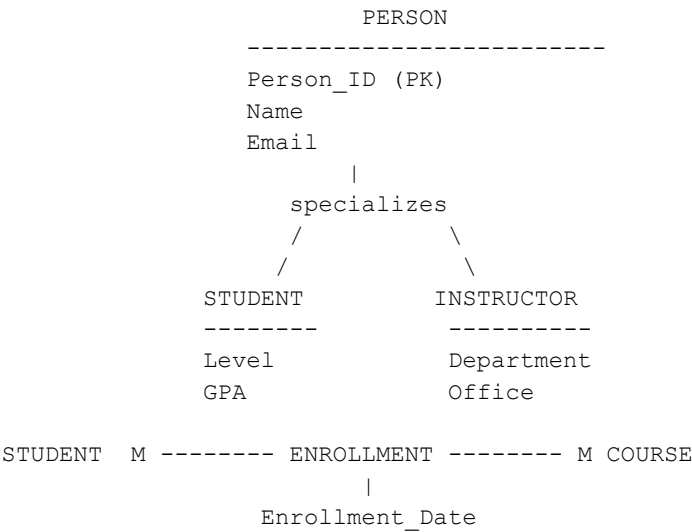
COURSE attributes
Course_ID  -> identifies Course
Course_Name-> describes Course
Credits    -> describes Course

ENROLLMENT attributes
Student_ID -> references Student
Course_ID  -> references Course
Enrollment_Date -> describes the enrollment

Attribute dependency:
Student_ID -> Name, Email
Course_ID  -> Course_Name, Credits
(Student_ID, Course_ID) -> Enrollment_Date
```

The ARD view emphasizes which attributes describe each object and which attributes depend on identifiers. This helps when deciding table columns, keys, and normalization rules.

C. EERD representation



The EERD can model Student and Instructor as specialized subtypes of Person. Both inherit common attributes such as Person_ID, Name, and Email, while each subtype has its own specific attributes.

1.6 When to Use Each

Diagram	When it is useful
ERD	Early and middle stages of database design, requirements analysis, communication with stakeholders, and identifying entities/relationships.
ARD	When detailed attribute dependencies, keys, and column-level organization need more attention.
EERD	Large or complex systems where inheritance, subtypes, specialization, or generalization are important.

Key idea to remember: ERD answers 'What entities exist and how are they related?', ARD emphasizes 'What attributes describe the data and depend on what?', and EERD answers 'What additional business meaning exists beyond a normal ER model?'

Task 2: Database Structures

2.1 What is a Database Structure?

A database structure is the way data is organized, stored, related, and accessed inside a database system. Different structures are designed for different workloads. The choice depends on factors such as data relationships, query patterns, consistency requirements, scale, and latency.

2.2 Major Database Structures

Structure	How data is organized	Advantages	Limitations	Examples / Uses
Hierarchical	Tree: one parent can have many	Simple navigation, predictable	Rigid; difficult for many-to-many	Legacy systems, organizational

	children.	relationships.	relationships.	structures.
Network	Records connected through sets/pointers; supports multiple parent-child relationships.	Handles complex relationships better than hierarchical.	Complex design and maintenance.	Legacy enterprise and telecom-style systems.
Relational	Tables made of rows and columns connected by keys.	Strong integrity, mature SQL, transactions, powerful joins.	Schema changes can require planning; horizontal scaling can be more complex.	Banking, ERP, CRM, e-commerce.
Object-Oriented	Stores objects with state and behavior similar to programming languages.	Natural mapping to object-oriented applications; supports inheritance.	Less common than relational systems; interoperability can be harder.	Engineering/scientific applications, specialized software.
Key-Value	A unique key maps directly to a value.	Very fast lookup; easy horizontal scaling.	Limited complex querying and relationships.	Caching, sessions, shopping carts.
Document	Records stored as JSON/BSON-like documents.	Flexible schema; nested data; developer-friendly.	Joins and strong relational constraints may be weaker or handled differently.	Content management, catalogs, user profiles.
Wide-Column	Rows are grouped into column families; columns can vary by row.	High scalability and throughput for large distributed workloads.	Data modeling is query-driven; not ideal for arbitrary relational queries.	IoT, telemetry, large-scale operational data.
Graph	Nodes represent entities and edges represent relationships.	Excellent for highly connected data and relationship traversal.	Can be unnecessary for simple tabular workloads.	Social networks, fraud detection, recommendations.
Time-Series	Data points organized around time, often with timestamps and measurements.	Efficient time-based storage and aggregation.	Specialized for temporal workloads.	IoT sensors, monitoring, finance, industrial telemetry.

2.3 Simple Examples

Relational:

```
STUDENT(Student_ID, Name, Email)
COURSE(Course_ID, Name)
ENROLLMENT(Student_ID, Course_ID)
```

Key-Value:

```
"user:1001" -> {"name":"Mona","role":"student"}
```

Document:

```
{
  "student_id": 1001,
  "name": "Mona",
  "courses": ["DB", "AI"]
}
```

Graph:
(Student:1001) --ENROLLED_IN--> (Course:DB)

A major lesson is that there is no universally best database structure. A system should use the structure that matches its access patterns and business requirements. Modern architectures may combine multiple database types.

Task 3: SQL Advantages & Disadvantages

3.1 Advantages of SQL / Relational Databases

1. Strong data integrity: primary keys, foreign keys, constraints, and validation rules help keep data correct.
2. ACID transactions: atomicity, consistency, isolation, and durability provide reliable transaction processing.
3. Powerful querying: SQL supports filtering, joins, grouping, aggregation, subqueries, and analytical functions.
4. Reduced redundancy through normalization: related data can be stored once and referenced using keys.
5. Mature ecosystem: relational databases have existed for decades and have large communities, tooling, and documentation.
6. Good fit for structured data: predefined schemas work well when business rules and data relationships are clear.
7. Excellent for transactional workloads: banking, orders, payments, inventory, and similar systems need reliable multi-step transactions.

3.2 Disadvantages

8. Schema rigidity: major changes to a predefined schema require planning and migrations.
9. Horizontal scaling can be more complex: traditional relational systems often rely heavily on vertical scaling, although modern systems also support sharding and distributed approaches.
10. Complex joins can become expensive: queries across many large tables may require careful indexing and optimization.
11. Less natural for highly unstructured or rapidly changing data: JSON, logs, media metadata, and irregular records may fit document or other NoSQL models better.
12. Operational complexity at very large scale: replication, partitioning, sharding, backup, and high availability can require specialist skills.
13. Potential over-normalization: highly normalized designs may require many joins, which can complicate some read-heavy applications.

3.3 SQL vs NoSQL

Aspect	SQL	NoSQL
Data structure	Relational tables, rows, columns, predefined schema.	Document, key-value, graph, or wide-column models; often flexible schema.
Scalability	Traditionally strong vertical scaling; horizontal scaling is possible but may require more design effort.	Often designed for horizontal scaling across nodes.
Flexibility	Structured schema and strong relationships.	Flexible models and easier schema evolution for many

		workloads.
Querying	Standardized SQL with joins and rich relational operations.	Varies by database type; queries are usually model-specific.
Transactions	Strong ACID support is a major strength.	Varies; many systems support transactions, but consistency models differ.
Typical uses	Banking, ERP, CRM, orders, inventory, financial transactions.	Large-scale web apps, flexible content, high-volume events, caching, graphs, IoT.

3.4 When Would You Choose Each?

SQL is a better choice for a banking transfer system because account balances and transfers require strong consistency and reliable multi-step transactions. NoSQL can be a better choice for a large-scale product catalog or event platform where records vary in structure and the system must scale horizontally across many nodes.

Task 4: OLTP vs OLAP

4.1 Definitions

OLTP (Online Transaction Processing) is designed for day-to-day operational transactions such as purchases, payments, bookings, and account updates. OLAP (Online Analytical Processing) is designed for complex analysis over large amounts of current and historical data.

Aspect	OLTP	OLAP
Purpose	Run daily business operations.	Analyze data and support decisions.
Data	Mostly current, detailed operational data.	Large historical and aggregated datasets.
Operations	INSERT, UPDATE, DELETE, simple SELECT queries.	Large SELECT queries, joins, aggregations, trends, drill-down.
Query style	Short and simple.	Complex and read-intensive.
Users	Customers, cashiers, bank tellers, operational staff, applications.	Business analysts, data scientists, managers, BI teams.
Performance goal	Very fast transaction response.	Efficient analytical query processing.
Schema	Usually normalized relational design.	Often dimensional models such as star or snowflake schemas.
Example	Online checkout or ATM transaction.	Monthly sales dashboard by region, product, and time.

4.2 Real-World Example

In an online store, OLTP records a customer's order, payment, shipping address, and inventory update immediately. Later, data from OLTP systems can be loaded into an OLAP warehouse. Analysts can then ask questions such as: 'Which product category produced the highest revenue by quarter and region during the last two years?'

4.3 Why a Company Needs Both

A company needs OLTP to operate the business and OLAP to understand and improve the business. Keeping heavy analytical queries away from the transactional database helps protect operational performance while giving analysts access to historical data.

Task 5: The 20 Vs of Big Data

Important note: there is no single universally accepted list called 'the 20 Vs'. The literature has proposed 3, 5, 7, 10, 13, 20+, and even much larger V-taxonomies. For this assignment, the following 20-V set is used as a practical academic synthesis of commonly discussed characteristics. The core ideas are widely recognized, while some extended Vs vary by source and application domain.

V	Meaning	Example	Why important
Volume	The amount of data generated and stored.	A social platform stores billions of posts, images, and interactions.	Requires scalable storage and distributed processing.
Velocity	The speed at which data is generated, transmitted, and processed.	IoT sensors send measurements every second.	Requires streaming ingestion and near-real-time processing.
Variety	The diversity of formats and sources.	Healthcare systems combine tables, notes, images, signals, and PDFs.	Requires integration and processing of structured, semi-structured, and unstructured data.
Veracity	The trustworthiness and reliability of data.	A sensor may report incorrect readings because of calibration problems.	Poor veracity can produce unreliable analytics and AI results.
Value	The useful business or operational benefit extracted from data.	E-commerce recommendations turn customer behavior into useful recommendations.	Large datasets matter only when they create actionable value.
Variability	Changes in data meaning, distribution, or patterns over time.	Customer behavior can change during holidays.	Models and pipelines must handle changing distributions.
Validity	Whether data is accurate for its intended purpose and follows expected rules.	A date field should contain a valid date.	Invalid data can break pipelines and reduce model accuracy.
Volatility	How quickly data loses usefulness or needs to be retained/deleted.	Real-time sensor data may be useful immediately but less useful months later.	Helps define retention, storage, and lifecycle policies.
Visualization	The ability to present complex data in understandable visual form.	A bank dashboard shows fraud rates by region.	Good visualization turns large datasets into decisions.
Vulnerability	Exposure to security, privacy, or unauthorized-access risks.	Healthcare records can contain sensitive patient information.	Big-data systems need access control, encryption, monitoring, and governance.
Vocabulary	The terms, labels, and meanings used to describe data.	Different hospital systems may use different names for the same measurement.	Shared vocabulary improves interoperability and data understanding.
Vagueness	Ambiguity or imprecision in data or meaning.	A customer review may say a product is 'good' without a measurable definition.	Ambiguous data can complicate classification and analytics.
Venue	The source, context, or	Social media, mobile apps,	Source context affects

	location from which data originates.	sensors, and websites generate data in different contexts.	interpretation, integration, and governance.
Viscosity	The difficulty or friction involved in moving, integrating, cleaning, or processing data.	A legacy system may make data extraction slow and difficult.	High viscosity can create pipeline bottlenecks.
Virality	How quickly data or information spreads through networks.	A social-media post can be shared by millions of users quickly.	Fast propagation affects infrastructure load, monitoring, and analytics.
Viability	Whether data is sufficiently useful and relevant for a particular purpose.	A weather dataset may be viable for forecasting but not for a medical model.	Prevents teams from spending resources on unsuitable data.
Vitality	How alive, active, or continuously changing a data source is.	Live patient monitoring data changes continuously.	High-vitality sources often require continuous ingestion and monitoring.
Verbosity	The amount of detail or redundancy contained in data.	Application logs may record many repetitive events.	Excessive verbosity increases storage and processing costs.
Verification	The process of checking whether data comes from a valid source and meets required rules.	A bank verifies transaction records against trusted systems.	Verification supports trust, compliance, and data quality.
Versatility	The ability of data and systems to support multiple uses or workloads.	A customer-event dataset may support analytics, recommendation models, and fraud detection.	Versatility increases the return on data investments.

Task 6: Data Engineering Tools Research

Data engineering uses different tools for ingestion, transformation, processing, streaming, orchestration, storage, and warehousing. A real production stack normally combines several tools rather than relying on one tool for everything.

Tool	Official Website	Category	Type	Main Purpose	Key Features	Languages / Integrations	Common Use Cases
Apache Kafka	https://kafka.apache.org/	Streaming / Ingestion	Open Source	Distributed event streaming and high-throughput pipelines.	Topics/partitions; durable event storage; consumer groups; scalable clusters.	Java/Scala ecosystem; connectors and integrations with databases, cloud services, Spark, Flink, and more.	IoT events, clickstreams, payments, logs, real-time pipelines.
Apache Airflow	https://airflow.apache.org/	Orchestration	Open Source	Author, schedule, and monitor workflows.	Python DAGs; scheduling; retries; dependencies;	Python; operators/integrations for databases, cloud platforms,	ETL/ELT orchestration, scheduled pipelines

					monitoring .	Spark, Kubernetes, etc.	, batch workflows.
Apache Spark	https://spark.apache.org/	Distributed Processing	Open Source	Large-scale batch and distributed data processing.	DataFrames/Spark SQL; distributed execution; MLlib; streaming support.	Python/PySpark, Scala, Java, SQL, R; many storage/cloud integrations.	ETL, big-data transformation, analytics, feature preparation.
Apache Flink	https://flink.apache.org/	Stream / Batch Processing	Open Source	Stateful processing of bounded and unbounded data streams.	Event-time processing; state management; exactly-once guarantee; low latency.	Java, SQL, Python APIs; Kafka and many connectors.	Real-time analytics, event-driven applications, fraud detection, IoT.
Apache NiFi	https://nifi.apache.org/	Data Ingestion / Flow	Open Source	Automate and manage data flows between systems.	Visual flow design; routing; transformation; back pressure; data provenance.	Processors/connectors for APIs, files, databases, Kafka, cloud systems, etc.	Data ingestion, routing, system integration, ETL-style flows.
dbt	https://www.getdbt.com/	Transformation / Analytics Engineering	Commercial / SaaS + open tooling	Transform warehouse/lakehouse data using SQL and software-engineering practices.	SQL models; tests; documentation; lineage; version control/CI workflows.	SQL; integrates with Snowflake, BigQuery, Redshift, Databricks and many others.	ELT transformations, data modeling, testing, analytics engineering.
Data bricks	https://www.databricks.com/	Lakehouse / Processing / Engineering	Cloud / Commercial	Unified platform for data engineering, analytics, and AI.	Spark-based processing; lakehouse storage; workflows/jobs; notebooks; ingestion and transformation.	Python, SQL, Scala, R; cloud storage and many data-source integrations.	ETL, lakehouse, streaming, ML data preparation, BI.
Google BigQ	https://cloud.google.com/bigquery	Data Warehouse	Cloud / SaaS	Fully managed analytics	Serverless architecture; SQL	SQL, Python APIs, BI tools,	Enterprise analytics

query				data warehouse.	analytics; scalable storage/compute; integrations with Google Cloud.	Dataflow, Pub/Sub, Airflow and others.	, BI, historical analysis, ML analytics.
Amazon Redshift	https://aws.amazon.com/redshift/	Data Warehouse	Cloud / SaaS	Cloud data warehouse for large-scale analytics.	SQL analytics; columnar storage; scalable compute; integration with AWS data services.	SQL; AWS ecosystem including S3, Glue, Kinesis, SageMaker and more.	Data warehousing, BI, analytics, lake/warehouse workloads.
Snowflake	https://www.snowflake.com/	Data Warehouse / Data Cloud	Cloud / SaaS	Managed platform for data warehousing, engineering, sharing, and analytics.	Separation of storage/compute; SQL; data sharing; scalable workloads.	SQL, Python/Java/Scala integrations, BI tools, cloud storage and connectors.	Enterprise warehouse, data sharing, analytics, data engineering.
AWS Glue	https://aws.amazon.com/glue/	ETL / Data Integration	Cloud / SaaS	Serverless discovery, preparation, movement, and integration of data.	ETL; Data Catalog; visual pipelines; serverless scaling.	Python/Scala; Spark; AWS S3, RDS, Redshift, Kafka, Kinesis and many connectors.	Data lakes, ETL, cataloging, analytics and ML data preparation.
Azure Data Factory	https://azure.microsoft.com/products/data-factory	ETL / Orchestration / Integration	Cloud / SaaS	Cloud data integration and pipeline orchestration.	Pipelines; visual data flows; scheduling/triggers; many connectors.	Azure services, SQL Server, Oracle, Databricks, Synapse, SSIS and many sources.	Cloud ETL, hybrid integration, scheduled data movement.

6.1 Example Data Engineering Stack

Data Sources

```
|
+--> APIs / Databases / IoT Sensors / Applications
```

```
|
```

```
v
```

Ingestion: Apache Kafka / Apache NiFi

```
|
```

```

    v
Processing: Apache Spark / Apache Flink
    |
    v
Orchestration: Apache Airflow
    |
    v
Transformation: dbt
    |
    v
Storage / Warehouse: Data Lake + BigQuery / Snowflake / Redshift
    |
    v
BI / ML / Applications
```

The exact stack depends on requirements. For example, Kafka is strong for event streaming, Airflow for workflow orchestration, Spark for distributed processing, and BigQuery/Snowflake/Redshift for analytical warehousing.